

DupliSense AI - Cross-Team Duplicate Work Detection & Reuse Recommendation Platform

PHASE I REPORT

Submitted by

GAYATHRI V R– 230701090

GOKUL ANAND B – 230701093

HARSHITA M A – 230701112

JYOSHNA K – 230701132

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

September 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled “**DupliSense AI - Cross-Team Duplicate Work Detection & Reuse Recommendation Platform**” is the bonafide work of **GAYATHRI V R (230701090), GOKUL ANAND B (230701093), HARSHITA M A (230701112), JYOSHNA K (230701132)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. Manoranjini M.E, Ph.D,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Dr.N. DuraiMurugan M.E, Ph.D,
Associate Professor
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

DupliSense AI is a web-based intelligent system developed to recognize duplicate and similar software projects and encourage reuse of existing software assets. Such platform works as an enterprise project information collection and analysis system of project name description problem statement, project objectives and technology used. Natural Language Processing (NLP) technique is used to clean and process the project information collected from various project sources and auto-matically ready to similarity analysis.

In what I just said system Term frequency inverse document frequency (TF-IDF) is applied to transform project information into feature vectors and Cosine Similarity is used to calculate the similarity between projects. After calculating the results of similarities, the project can be categorized and re-usable artifacts, such as source code APIs database structure, UI components documentation could be discovered.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N.S MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. Manoranjini, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, Dr. P. Muneeshwari, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, Mr. Karthik, B.E., M.E., Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

GAYATHRI V R 230701090

GOKUL ANAND B 230701093

HARSHITA M A 230701112

JYOSHNA K 230701132

TABLE OF CONTENTS

ABSTRACT	iii
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	ix
1. INTRODUCTION	1
1.1 GENERAL	1
1.2 OBJECTIVE	2
1.3 EXISTING SYSTEM	3
1.4 PROPOSED SYSTEM	3
2. LITERATURE SURVEY	5
2.1 LITERATURE REVIEW	5
2.2 COMPARISON AND DISCUSSION	9
2.2.1 RESEARCH GAP	11
2.2.2 PROPOSED CONTRIBUTION/NOVELTY	12
2.3 CONCLUSION	13
2.4 RESEARCH QUESTIONS	13
3. SYSTEM DESIGN	16
3.1 SYSTEM ARCHITECTURE	16
3.2 SYSTEM REQUIREMENTS	18
3.2.1 SOFTWARE REQUIREMENTS	18
3.2.2 DATASET REQUIREMENTS	21
3.3 DATABASE DESIGN	22

3.4 DEPLOYMENT ARCHITECTURE	24
4. PROJECT DESCRIPTION	26
4.1 METHODOLOGY	26
4.2 MODULES	26
4.2.1 DATA MANAGER PACKAGE	27
4.2.2 NLP AND SIMILARITY PACKAGE	28
4.2.3 REUSE RECOMMENDATION PACKAGE	29
4.2.4 USER MANAGEMENT PACKAGE	29
4.2.5 UI PACKAGE	30
4.2.6 ROLE-BASED FRONTEND PORTALS	31
5. IMPLEMENTATION AND RESULT DISCUSSION	32
5.1 IMPLEMENTATION RESULTS	32
5.2 IMPLEMENTATION OF DATA PREPROCESSING	34
5.1.2 IMPLEMENTATION OF DATA TOKENIZATION	35
5.1.3 IMPLEMENTATION OF DATA EMBEDDING	36
5.2 DISCUSSION	39
6. CONCLUSION AND FUTURE WORK	40
6.1 CONCLUSION	40
6.2 FUTURE WORK	41
REFERENCES	43

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
3.1	Backend software stack and libraries	19
3.2	Frontend software stack and libraries	20
3.3	System requirements	21
3.4	Database entities and their purpose	22

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	System architecture diagram	17
4.1	Role Based Login	31
5.1	Developer Dashboard	33
5.2	Manager Dashboard	33
5.3	Admin Dashboard	34
5.4	Data Extraction	35
5.5	Semantic Similarity and Code Scan Results	38
5.6	Semantic Vector Match and Code Overlap	38

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
FK	Foreign Key
HITL	Human-in-the-Loop
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
RQ	Research Question
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language

CHAPTER 1

INTRODUCTION

1.1 GENERAL

In software developing organisations and academia there are usually many projects which one organisation is carrying on to overcome different problems. The higher the number of projects is, the greater the unavoidable probability that teams working separately from each other will unknowingly develop similar projects in similar areas features using similar technologies or similar ways of implementation. And, similar projects need to redo the work that a team has already invested on the software development and that new projects should be making a previous use of the available software assets.

DupliSense AI a smart online tool to identify duplicate and overlapping projects. It takes in all information related to a project (a project title description problem purpose technologies) and works with Natural Language Processing (NLP) techniques to identify similarity between various projects. TF-IDF (Term Frequency Inverse Document Frequency) has been used to transform project information into vectors, and the Cosine Similarity algorithm is used by the system to obtain the degree of similarity between projects, based on these vectors. Finally, the system proposes projects, which are similar or dissimilar to some extent, as well as the corresponding similar tools (code APIs databases UI documentation) and expertise, to resolve the redundancy issue.

The application allows three kinds of modes to view the data (Developers Manager Administrator). And, DupliSense AI shows dashboards and reports to display that

data to their users. This application was created using React, Vite and Django on the front-end side and both the back-end and the NLP functionality used the SQLite and the scikit-learn libraries.

1.2 OBJECTIVE

The main goal of DupliSense AI is to provide an intelligent system for automatically detecting the similar and duplicate software projects to use the reusable resources from previous software projects. So, following goals are set for DupliSense AI system:

- To have one platform where the project descriptions are stored and edited.
- To investigate the project appraises description goal problem statement and the technology used using NLP;
- For computing similarity metrics between the software project using; TF-IDF and Cosine Similarity
- To locate and automatically flag similar or not identical project(s);

An algorithm used for dividing projects based on how similar they are:

- To recommend the similar projects' resources (code API database, UI parts and documents) for reusing;
- Setting up dashboards to visualize the similarity matrix and project re-use.
- To converge wasted effort during development and take the full advantage of software reuse;

1.3 EXISTING SYSTEM

Using our current procedure, the project duplication had to be analyzed manually. The duplication is checked by putting the project title description objective, technology and functions side to side. When there are too many projects to analyze, the procedure is too long and the manual comparison of these projects makes possible to forget similar projects in the description.

And, the proposed approaches currently do not give the user a reference value of the level of similarity of projects; that is, a measure of how similar projects are. The current approaches also do not automatically propose resources that can be reused for implementation of the new project. It is because of these limitations that the work presented in this paper is developing a method that can compare projects automatically and suggest resources that can be reused.

1.4 PROPOSED SYSTEM

DupliSense AI is suggested as an automatic duplicate/overlapping software projects identification system; the system maintains information about the projects in a common database and applies NLP algorithms to examine the project documents.

The information about the projects is converted into TF-IDF vectors and the similarity between projects is obtained by the Cosine Similarity Algorithm. The system can find similar projects and categorize it per value obtained.

This system will leverage available resource from other existing systems like project source code, APIs and databases; UI/UX documentation knowledge etc. To separate and categorize Users, Managers and Admins, Role-Based Access Control will be used. Users can manage their own projects, view similar project and receive reuse suggestions and alerts. Managers will be able to view the nearest resources and

suggested similarities as well as to review the resources which are suggested for potential reuse. Admins will manage the system and set its configurations. Dashboards for Users, Managers and Admins will be provided for high level insight.

For the system were the frontend was developed with React and Vite, and the backend using Django REST structure. The system had SQLite as database management system, and the similarity calculations and the NLP tasks were made with scikit-learn. The proposed system offers an automated system for saving, comparing, and reusing projects resources.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews published work in the areas that DupliSense AI draws on: duplicate detection in software engineering, text similarity and information retrieval, semantic similarity using embeddings, software project similarity analysis, role-based access control, RESTful software architectures, and software reuse. Each research area is reviewed independently to understand the techniques, advantages, and limitations of existing approaches. Section 2.2 then discusses how these findings collectively influenced the design of the proposed DupliSense AI platform.

2.1 LITERATURE REVIEW

Research on duplicate detection has primarily focused on identifying semantically or textually similar items within large software repositories and issue-tracking systems. Nguyen et al. proposed a hybrid approach for duplicate bug report detection by combining information retrieval techniques such as TF-IDF with topic modeling using Latent Dirichlet Allocation (LDA) [1]. Their work demonstrated that textual similarity can be effectively used to identify duplicate reports in large-scale software repositories. This research directly influenced the adoption of TF-IDF as a core text-vectorization technique in DupliSense AI.

Xia et al. provided a comprehensive survey of Natural Language Processing techniques used in software engineering, covering approaches ranging from traditional bag-of-words and TF-IDF representations to word embeddings and deep learning models [2]. The study highlights the trade-off between computational complexity, interpretability, and similarity accuracy. For moderate-sized collections

of technical documents, traditional TF-IDF-based approaches remain useful because of their computational efficiency and interpretability. This supports the use of TF-IDF as an efficient baseline for project similarity analysis in DupliSense AI.

He, Zibran, and Roy investigated duplicate requests in online software service desks using TF-IDF and different similarity measures [3]. Their study compared cosine similarity, Jaccard distance, and Euclidean distance for identifying similar service requests. The results demonstrate the suitability of cosine similarity for comparing variable-length textual documents because it focuses on the orientation of vectors rather than their absolute magnitude. This finding supports the use of cosine similarity in DupliSense AI for comparing project descriptions that may differ significantly in length.

Gomaa and Fahmy examined text similarity using cosine similarity and n-gram analysis [4]. Their work demonstrates that incorporating bigrams and trigrams can improve text matching by capturing meaningful combinations of words rather than considering individual terms independently. This is particularly relevant to technical project descriptions where phrases such as "machine learning", "REST API", "face recognition", or "role-based authentication" carry more meaning as combined terms. Based on this finding, DupliSense AI uses an extended TF-IDF representation with unigram, bigram, and trigram features.

Semantic text representation has also been investigated through deep learning approaches. Reimers and Gurevych introduced Sentence-BERT (SBERT), which generates sentence-level embeddings suitable for semantic similarity tasks [5]. SBERT provides stronger semantic understanding than conventional lexical approaches and can identify relationships between sentences even when they use different vocabulary. However, embedding-based approaches generally introduce

additional computational requirements compared with traditional TF-IDF methods. Therefore, DupliSense AI currently uses TF-IDF for efficient project comparison while considering sentence-embedding approaches such as SBERT as a future enhancement.

Research has also investigated access control mechanisms required for secure multi-user software systems. Sandhu et al. introduced the foundational Role-Based Access Control (RBAC) model, in which permissions are assigned to roles and users are assigned to those roles [6]. This model provides a structured mechanism for controlling access to system resources according to organizational responsibilities. DupliSense AI applies this principle through three major roles: Developer, Manager, and Admin. Developers can submit and review project information, Managers can perform project-approval and access-control operations, and Administrators have broader governance and system-management privileges.

Huang compared multiple document similarity techniques including TF-IDF, Word2Vec, Doc2Vec, and BERT-based representations [7]. The study highlights the continued usefulness of TF-IDF for structured technical documents because technical terminology, framework names, APIs, and domain-specific keywords can provide strong indicators of document similarity. This observation supports the use of TF-IDF in DupliSense AI, particularly for projects containing identifiable technology stacks and technical terminology.

Ortu et al. specifically investigated the automatic identification of similar software projects in software repositories using project README files and metadata [8]. Their approach combines textual similarity with technology-stack overlap rather than relying exclusively on project descriptions. This work is particularly relevant to DupliSense AI because software projects can appear textually different while still

using similar technologies and addressing similar functional requirements. The study therefore supports the proposed combination of project-description similarity and technology-stack similarity in the DupliSense AI similarity pipeline.

Software-system architecture is another important consideration for implementing the proposed platform. Pautasso, Zimmermann, and Leymann presented REST API design principles covering resource-oriented URLs, HTTP methods, response representations, and service communication [9]. These principles provide a foundation for designing modular backend services. DupliSense AI follows RESTful design principles for communication between its frontend and backend components, including project management, similarity analysis, authentication, and approval-related operations.

Finally, software reuse research emphasizes the importance of identifying existing software assets before developing new solutions. Krueger's survey of software reuse strategies discusses component-based, service-oriented, and knowledge-based reuse approaches [10]. The study highlights automated identification of reusable software assets as an important factor in improving organizational reuse. This principle forms the central motivation of DupliSense AI, where similarity detection is followed by reuse recommendations so that teams can identify existing projects, components, technologies, or approaches before duplicating development effort.

Overall, the reviewed studies establish that duplicate detection, textual similarity, semantic representation, secure access control, project similarity analysis, and software reuse are individually well-established research areas. However, these capabilities are generally investigated separately or within specific application domains.

2.2 COMPARISON AND DISCUSSION

The reviewed literature establishes several design principles that directly influenced DupliSense AI. First, research on duplicate bug reports and service requests demonstrates that TF-IDF combined with cosine similarity provides an efficient and interpretable method for comparing textual information [1], [3]. This motivated the adoption of TF-IDF and cosine similarity as the primary similarity mechanism in DupliSense AI.

Second, the literature indicates that considering only individual words can be insufficient when analysing technical project descriptions. N-gram-based approaches capture meaningful combinations of terms and therefore provide additional contextual information [4]. DupliSense AI consequently extends conventional TF-IDF using unigram, bigram, and trigram features so that technical phrases and multi-word concepts can contribute to similarity computation.

Third, semantic embedding research demonstrates that transformer-based models such as Sentence-BERT can capture deeper semantic relationships between texts [5]. However, the computational requirements of embedding-based approaches make them less lightweight than traditional TF-IDF approaches. For the current implementation, DupliSense AI prioritizes an efficient and explainable similarity mechanism while retaining semantic embeddings as a possible future enhancement.

Fourth, research on software-project similarity indicates that textual information alone does not fully represent project similarity. Combining project descriptions with metadata and technology-stack information can provide a more meaningful representation of software projects [8]. DupliSense AI therefore considers both descriptive project content and technology information when generating similarity results.

The literature on RBAC additionally demonstrates the importance of controlling system functionality according to organizational responsibilities [6]. This principle is reflected in DupliSense AI through separate Developer, Manager, and Admin roles. Similarly, REST architectural principles provide a foundation for exposing modular backend functionality through structured APIs [9].

The reviewed software-reuse literature further establishes that similarity detection becomes more valuable when it leads to actionable reuse rather than simply reporting a similarity score [10]. DupliSense AI therefore extends the conventional duplicate-detection concept by connecting similarity analysis with project discovery and reuse recommendations.

Where the reviewed systems differ most from DupliSense AI is in their scope of integration. Duplicate bug-report systems primarily focus on issue descriptions, text-similarity studies focus on document comparison, SBERT focuses on semantic sentence representation, RBAC research focuses on authorization, REST research focuses on service architecture, and software-reuse studies focus on identifying reusable assets. These approaches do not generally combine project similarity analysis, technology-stack comparison, role-based governance, approval workflows, and reuse recommendations into one organizational platform.

DupliSense AI addresses this integration requirement by providing a unified workflow in which project information can be submitted, processed, compared with existing projects, reviewed according to user roles, and converted into actionable reuse recommendations.

2.2.1 RESEARCH GAP

As seen in Section 2.1, text similarity detection and duplicate detection is an already mature research area in software engineering. Existing similarity detection methods of text include TF-IDF [1]-[5], [7], cosine similarity, topic modeller, n-gram, and transformer-based embedding [7] [3]. But, most of existing techniques found just calculate the similarity between texts but did not introduce to a full project-management and software-reuse cycle.

The automatic comparison between software projects in literature is mostly only for software-project comparison based on README files and project information [8]. The complete workflow of software project submission, software-image comparison, factor comparison by technology-stack, access control, management approval and software reuse recommendation has yet been minimized.

The research on RBAC gives best practice of controlling the access to resources within an organization with roles of users [6]. and REST architecture brings best practice of building software services with modularity and scalability [9]. But the two concepts are studied largely in isolation within duplicate-project detection and software reuse.

As the reviewed literature, there is a requirement for a common platform where the new project proposals can be matched automatically to the existing projects in the company repository (by finding candidate duplicate or similar projects), to identify the technologies used in the existing projects to compare and to make recommendations on software reuse with role-based access control (RBAC) and management approval.

In the broad stance, the concept of the Dupli Sense AI, which is to make provision of an integrated platform for project record management, the calculation of text similarity, the comparison of technology-stack, access control, supervisor approval, and software-reuse recommendation, is presented.

2.2.2 PROPOSED CONTRIBUTION/NOVELTY

The main contribution of the Dupli-Sense- AI is that it is integrating the various existing tasks of project similarity detection and software re-usage in a single one into a platform that can be used in an organization. Rather than designing a new similarity detection algorithm, the work proposed has concentrated on placing existing techniques into a full project-management and re-usage workflow.

The proposed system takes the project proposal and the project documents supported by developers, and extracts project data. The system compares projects with the existing projects in the repository. This uses the TF-IDF representation of the unigram, bigram, and trigram features of project descriptions, and compares the representations about cosine similarity, plus the technology stack features for identifying related projects with similar technical capabilities.

The similarity comparison results are utilized to produce recommends for software re-use and locate replicated and similar applications.

The recommendations are meant to expose and showcase projects components technologies or knowledge that can be reused. Architectures are intended to have role-based responsibility for Developers, Managers, and System Administrators such that access to sensitive project data can be constrained.

The main contribution of this system is in a fusion of already existing approaches: detection of similar projects, technology stack analysis, reuse suggestion, and role-based control, in a single tool.

2.3 CONCLUSION

The relevant literature validates the key architectural decisions in DupliSense AI. TF-IDF is a scalable, effective and transparent representation for technical project descriptions [1], [2], [7]; cosine similarity is an effective match metric for variable-length textual documents [3]; n-gram fingerprinting enhances the expression of technical terms and multi-word expressions [4]; and work on semantic embedding offers the potential for deeper semantic matching [5].

Existing literature on software-project similarity all agree on the advantage of blending textual project data with the features specific to technology-stacks [8] 6 9, 10: principles of RBAC enable determining access rights based on organizational roles [6], REST principles drive design of a modular back-end [9], researches on software-reuse paves way for the rest-rest synergy [10].

While these individual techniques are easily accomplished, there appears to be minimal proof through the literature reviewed of a single platform that encompasses project similarity detection, technology-stack comparison, role-based governance, approval process, and reuse recommendations. DupliSense AI seeks to fill this gap with an integrated approach to multiple functions transforming a duplicate-project similarity task into a matter of organization-wide project discovery and reuse.

2.4 RESEARCH QUESTIONS

The research gap and contribution proposed in Sections 2.2.1 and 2.2.2 is presented as this research questions

- RQ1: Can a project-similarity tool automatically recognize potentially similar or duplicate software projects based on project descriptions and supporting information?
- RQ2: Is TF-IDF using unigram, bigram and trigram feature combined with cosine similarity an efficient and interpretable approach of comparing technical project descriptions?
- RQ3: Is it feasible to utilize information about technology-stack, to enhance the task of recommending related software projects, over traditional content-based project description?
- RQ4: Is it feasible to incorporate similarity detection into a reuse-recommendation process in which developers are supported in finding projects components technologies, or methods in the past that might be applicable to the project at hand?
- RQ5: Could role-based access control of the project administrator, developer and manager maintain the intended separation of responsibilities and controlled access to information of the project?
- RQ6: What were the measured similarity detection performance characteristics (precision recall F measure, etc.) and processing time behaviours of the implemented DupliSense AI on the chosen evaluation dataset?

Questions RQ1- RQ5 relate to the architecture and functionality of the proposed platform and will be addressed by the system design and implementation presented in the next chapters. Question RQ6 is a research question which can only be answered empirically, by evaluating the platform with a set of project records and

measuring the similarity results. Quantitative performance values should only be included after experimenting with the implemented system and the evaluation set.

CHAPTER 3

SYSTEM DESIGN

This chapter describes DupliSense's overall architecture, the software and hardware environment it targets, its database design, and the way the system is packaged for deployment.

3.1 SYSTEM ARCHITECTURE

The architecture of the DupliSense AI determines the flow of the user interface, application server, database and similarity analysis. The system architecture as it can be defined is based on layers and every layer brings a value to the upper layer i.e. user interface. All the other layers communicate with each other via backend.

This front end is generated by React and Vite which supposed to be this presentation layer. It is most of the part enables login, uploading a project, project management, displaying similarity result, as repurposing suggestion, and dashboards. A user might have his own fronts, such as Developer Manager Administrator.

Setup is handed over by application layer; built using Django REST setup; manages user role management; Handles user authentication, process the data related to projects and other compute requests for similarity, manage recommendations and communicate with database.

The NLP, natural processing language has been used in process the project data. All the project information including project title description problem statement goal technology are processed by NLP and convert to TFIDF vector. Then use the Cosine Similarity for estimating the project data. The similarity index is then used to decide the value of that project.

The database layer utilized for storing the information about users, the data about projects, the results of similarities and recommendations of reuse are SQLite. All the result which has been done are collected from the backend and displayed on the front end.

The major flow of the system is:

User -> React/Vite -> Front-end Django-> REST configuration -> NLP & Similarity Module-> SQLite Database -> Results & Recommendations User

The architecture proposes a common setup to handle software projects and their comparison analysis. As well it can support the system proposed for redistributing recommendations supported the project similarities identified.

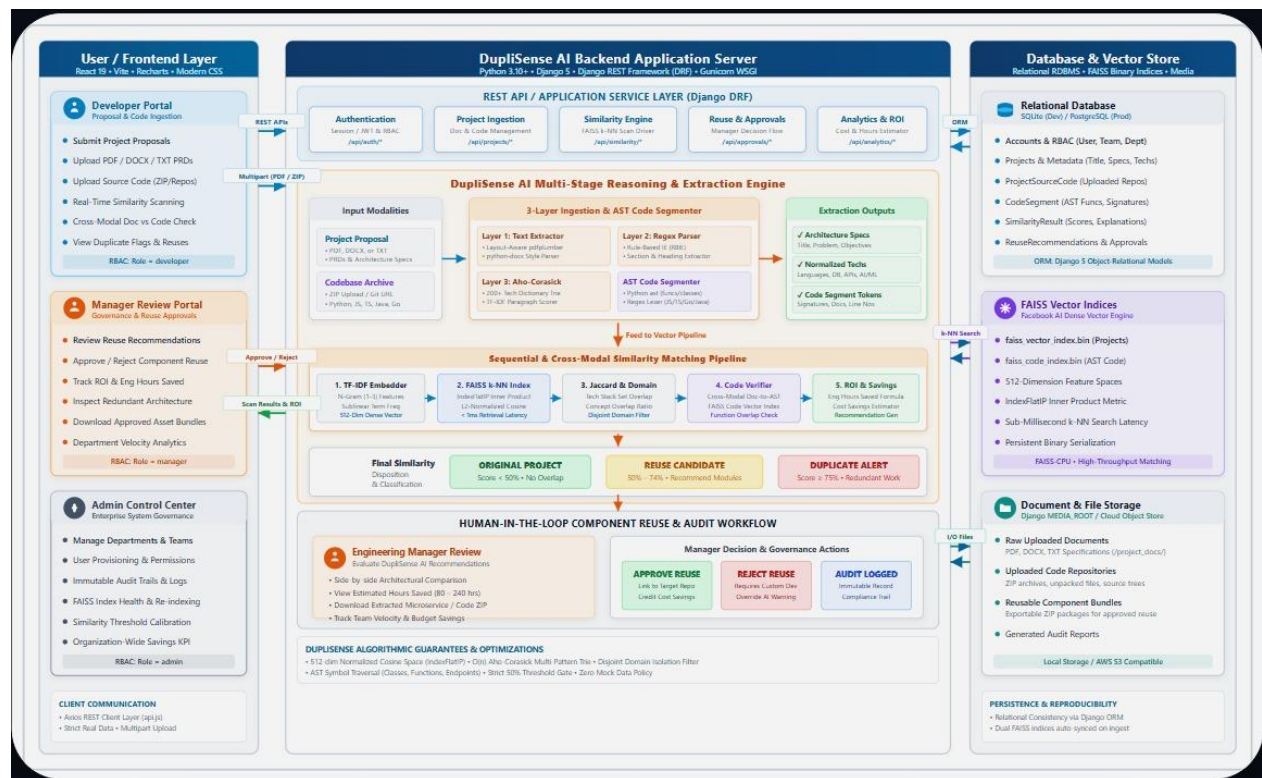


Figure 3.1. System architecture diagram

The diagram above is intended to show, at a glance: the three role-scoped frontends on the left; the FastAPI application server in the centre, split into its REST routers (auth, claims, investigator, admin) and its agentic core (the Intake graph and the Supervisor graph, each drawn as a small state machine); the external services the agentic core calls out to (the configured LLM provider, and PaddleOCR as a local fallback OCR engine); and the MySQL database on the right, grouped into its three logical clusters — supply-chain and warranty records, claims and agent-execution records, and identity/audit records. Arrows should show the asynchronous background-task hand-off between the claim-submission endpoint and the agent pipeline, since that hand-off is what allows the customer-facing request to return instantly while validation continues in the background.

3.2 SYSTEM REQUIREMENTS

The system requirements define the software, development tools, technologies, and data requirements needed to develop and operate DupliSense AI. Since the proposed system is completely software-based, the requirements mainly focus on the application framework, database, NLP libraries, and project data required for similarity analysis.

3.2.1 SOFTWARE REQUIREMENTS

The major software requirements of the system are:

The frontend provides the user interface, while Django REST Framework manages the backend services and APIs. SQLite is used for storing project and user information. The scikit-learn library is used for TF-IDF vectorization and Cosine Similarity calculation.

Component	Role
Python	Core programming language running the backend application server, AST lexical parsers, and vector matrix operations.
Django	Enterprise web framework providing Object-Relational Mapping (ORM), session handling, migration management, and admin governance.
Django REST Framework (DRF)	Serializes complex model instances into JSON, enforces request schema validation, and implements RBAC permission gates.
django-cors-headers	Cross-Origin Resource Sharing (CORS) handler regulating secure cross-origin HTTP communication between Vercel and Render.
Scikit-Learn	TF-Idf Vectorizer extracting sublinear term frequencies, (1,3)-word n-grams, and calculating pairwise cosine similarity matrices.
FAISS (faiss-cpu)	Facebook AI Similarity Search engine executing sub-millisecond \$k\$-NN cosine distance search over 512-dimensional embeddings.

NumPy	High-performance array operations for vector padding, dimensionality alignment (512-dim), and L2 vector normalization.
Pdfplumber	Layout-aware PDF parsing engine extracting raw architectural text, structured tables, and paragraph flows from uploaded PRDs

Table 3.1. Backend software stack and libraries

The frontend is a TypeScript single-page application built with Vite and React 18, styled with Tailwind CSS v4. Table 3.2 lists the key frontend dependencies, taken directly from the project's package.json.

Component	Role
React	Component-based reactive user interface rendering interactive developer, manager, and administrative dashboards.
Vite	High-performance build tool and development server providing instant Hot Module Replacement (HMR) and optimized Rollup bundling.
React Router DOM	Declarative client-side Single Page Application (SPA) routing with protected route guards enforcing RBAC authorization

Axios	Promise-based client managing asynchronous REST API calls, JWT authentication interceptors, and multipart form file upload streams
Recharts	Declarative SVG charting library rendering real-time ROI savings graphs, team efficiency metrics, and similarity score distributions
React Icons	Curated vector iconography (Lucide, Feather) for clear visual status indicators, modal dialogs, and navigation drawers.

Table 3.2. Frontend software stack and libraries

3.2.2 DATASET REQUIREMENTS

The system requires software project information for performing similarity analysis. The major project attributes include:

Component	Recommendation
CPU	4 or more cores (PaddleOCR inference and LLM I/O both benefit from parallelism)
RAM	8 GB minimum; 16 GB recommended once PaddleOCR's models are loaded
Storage	5 GB+ free (Python/Node dependency trees, PaddleOCR model weights, MySQL data directory)

GPU	Not required; PaddleOCR runs in CPU mode by default (FLAGS_use_mkldnn disabled for Windows/CPU compatibility)
Network	Outbound HTTPS access to the configured LLM provider's API (OpenAI, Gemini, or Anthropic)
Containerisation	Docker and Docker Compose for the one-command docker compose up --build deployment

Table 3.3. System requirements

3.3 DATABASE DESIGN

The database design of DupliSense AI is used to store and organize user information, project information, similarity analysis result, and Reuse information. The database serves as the common storage layer of the system to allow for communication with other system. Due to in reality DupliSense AI is a software-based web application, the choice of the database management system is SQLite.

The database is to store the user project relationship. Project related information like project title description problem statement, objectives and technology stack are the inputs for both NLP and similarity analysis module. Besides, the outcome of similarity analysis will also be stored in database so that user can monitor the project relationships.

Table	Purpose
Department	Purpose: Models top-level organizational business units (e.g., Engineering, R&D, Data Science) to establish organizational

	hierarchy, access scopes, and cross-departmental reuse tracking
Team	Groups developers and managers into specialized engineering squads within a department, facilitating squad-level duplicate alerts and collaborative component sharing
Project	Stores comprehensive project proposal specifications, architectural narratives, problem statements, and extracted multi-category technology stacks for vectorization and similarity matching
Project Source Code	Stores individual source code files unpacked from uploaded repository ZIP archives or Git repositories for AST structural parsing
Code Segment	Records pairwise similarity comparisons generated by the FAISS k -NN vector engine and Jaccard overlap matcher, persisting similarity percentages and RAG explanations
Similarity Result	Records pairwise similarity comparisons generated by the FAISS k -NN vector engine and Jaccard overlap matcher, persisting similarity percentages and RAG explanations
Recommendation	Represents an actionable recommendation generated when similarity exceeds the 50% threshold, proposing specific microservices, libraries, or APIs to reuse

Activity Log	Provides an immutable, system-wide audit trail recording user logins, proposal ingestions, similarity scans, and managerial approvals for enterprise compliance
--------------	---

Table 3.4. Database entities and their purpose

Foreign keys tie the supply-chain tables together through `serial_number` (customer, seller, `manufacturer_to_seller` and `claim_history` all reference `manufacturer.serial_number`) and `claim_history` additionally references `warranty.warranty_id` once a claim has been matched to a specific warranty record. The three agent-observability tables (`agent_runs`, `validation_results`, `fraud_signals`) are keyed by `claim_id` but are populated by application logic rather than SQLAlchemy relationships, which keeps the agent layer decoupled from the ORM's relationship graph and free to write execution traces without needing a live claim object in memory.

3.4 DEPLOYMENT ARCHITECTURE

The project provisioned containerization support with a Dockerfile for the frontend and backend, coordinated by a root-level Docker Compose configuration so that the entire stack can be launched with a single `docker compose up --build` command (or run locally with `npm run dev` and `python manage.py run server`). The frontend container hosts the production build of React 19 + Vite; the backend container hosts a Gunicorn WSGI server serving the Django 5 / Django REST structure application; the relational database service (backed by the developer's local SQLite instance) is PostgreSQL (with the backend's `DATABASE_URL` environment variable configured to connect to it) plus persistent binary storage for the FAISS vector indices (`faiss_vector_index.bin` and `faiss_code_index.bin`). Configuration most

critically the Django SECRET_KEY, ALLOWED_HOSTS, and CORS_ALLOWED_ORIGINS parameters are provided via a.env file (copied over from the committed.env.example) instead of being hard-coded. Django migrations (python manage.py migrate) and the automated seed_data manage command (python manage.py seed_data) are run against it to establish the 3NF schema, index baseline project vectors into FAISS, and populate the default developer manager admin department project, and component reuse recommendation records used throughout the evaluation and testing.

Frontend Web Application (Vercel):

- <https://duplisense-ai.vercel.app>

Backend REST API Server (Render):

- <https://duplisense-ai.onrender.com/api>

Backend Admin Governance Portal (Render):

- <https://duplisense-ai.onrender.com/admin>

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGY

The process of DupliSense AI involves stages for the acquisition of project information and for analysis, project similarity measurement and the suggestion of reuse projects.

The title description problem statement objectives technology stack, are entered and stored at the initial stage of project. The description and other textual information of project is perform preprocessing like stemming, tokenisation and more using NLP to get ready for similarity measure.

The data that has been processed is transformed into numerical feature vectors by TF-IDF. Cosine Similarity is used to find the similarity between projects and the score is used to retrieve and classify projects by their similarity.

If a high level of similarity is found, recommendations for alternative objects (e.g. code API database, UI, or documentation) that could be reused are created. The output is presented using the web app and the dashboards.

Also, the system implements the role-based access for Developers, Managers and Administrators. This approach offers a systematic scheme to identifying duplication of projects and assisting in reuse of existing software assets.

4.2 MODULES

DupliSense AI system is comprised of high-level packages, and each package is consisted of some modules which achieved a certain stage on project analyse work.

A high-level package can communicate with other high-level packages via backend services and database, and the project data flow from data collection stage, similarity analysis stage, reuse recommendation, and so on. The high-level package make the system much easier for maintain, update, and extend.

The major packages of the system are:

1. DATA MANAGER PACKAGE
2. NLP AND SIMILARITY PACKAGE
3. REUSE RECOMMENDATION PACKAGE
4. USER MANAGEMENT PACKAGE
5. UI Package

4.2.1 DATA MANAGER PACKAGE

Data Manager Package handles collecting storing accessing and managing of data related to software project. Data Manager Package is the first step in DupliSense's AI flow, as the quality and the completeness of the information about the project influences the idea of similarity. Data Manager Package brings necessary interfaces to input information of project, also communicating with Django REST system using SQLite as the database.

The data of the project consists of project title description problem description, project objectives and project technology. These are the parameters used to describe a project. These parameters are then passed to the NLP and Similarity

Package. The package supports the retrieval of existing projects so that a new project can be compared with any number of projects already stored in the system.

The Data Manager Package sorts and stores project data and enables updates to that data should the project change. It also gives the project data to the modules that need it for similarity analysis. By keeping an in one place all the project data, the package avoids users having to gather the information elsewhere before a project comparison.

4.2.2 NLP AND SIMILARITY PACKAGE

The NLP and Similarity Package is the main analytical component of DupliSense AI. It takes the textual data from the Data Manager Package and analyses it to determine how similar software projects are to each other.

The project title description problem statement, objectives and other textual information about the projects are handled using NLP techniques. These textual information are represented in the form of feature vectors using TF-IDF (Term Frequency- Inverse Document Frequency). It aims at rendering the terms based on the importance they give in the project information and helps capture the text features of individual projects.

Once all the features are extracted for the projects, Cosine Similarity between the generated project vectors is computed and similar projects are identified by finding the similarity score among all the projects. The similarity score would be indicative of the degree of similarity among the projects. The projects would be identified to be similar if they if they have similar goals functionalities description and technology requirements. The similarity results would be categorized under different levels of similarity.

This package employs scikit-learn to carry out the vectorization necessary as well as the similarity calculations required. The output of this package is then handed off to the Reuse Recommendation Package and Dashboard and Reporting Package.

4.2.3 REUSE RECOMMENDATION PACKAGE

The Reuse Recommendation Package uses the output of the NLP and Similarity Package to recommend potentially reusable resources from other similar projects. The objective of this package is to enhance the system outside naive duplicate detection to deliver valuable information for software reuse.

Where there is a strong match, the system will look at the project data that exists and offer suggestions about which contents might be used again. This could relate to source code APIs database structure, user interface documentation Know-how.

Finally the recommendations are passed to the relevant user(s) for approval. Managers can view the similarity results and recommendations and accept or decline the potential re-use options. This gives a more controlled approach for deciding if an existing resource should be reused on another project.

The package this way links the results of the similar analysis with the project development activities in practice. It can assist project teams to search for existing resources before re-implementation of the same features.

4.2.4 USER MANAGEMENT PACKAGE

Authentication and access control is provided by the User Management Package within the DupliSense AI platform. This package brings a number of levels of access given the nature of the user.

System allows to work with three main roles; Developer Manager Administrator. Developer can input and keep project information, view similarity result of projects. Manager can read out similarity results and control reuse recommendation. Administrator can see all system level information and observe all the project activities, similarity trend and statistics including reuse.

Role-Based Access Control (RBAC) restricts user access to only those functions needed for their role. The package integrates with your backend authentication & authorization to secure your project information and system operations.

4.2.5 UI PACKAGE

UI Package. This package offers the user interface of the DupliSense AI system as well as the interface layer between the users and the backend services. It is build on React with Vite and offers these user interactions: login, apply for a project, manage projects, similarity analysis, recommendations for reusing, dashboard visualisation.

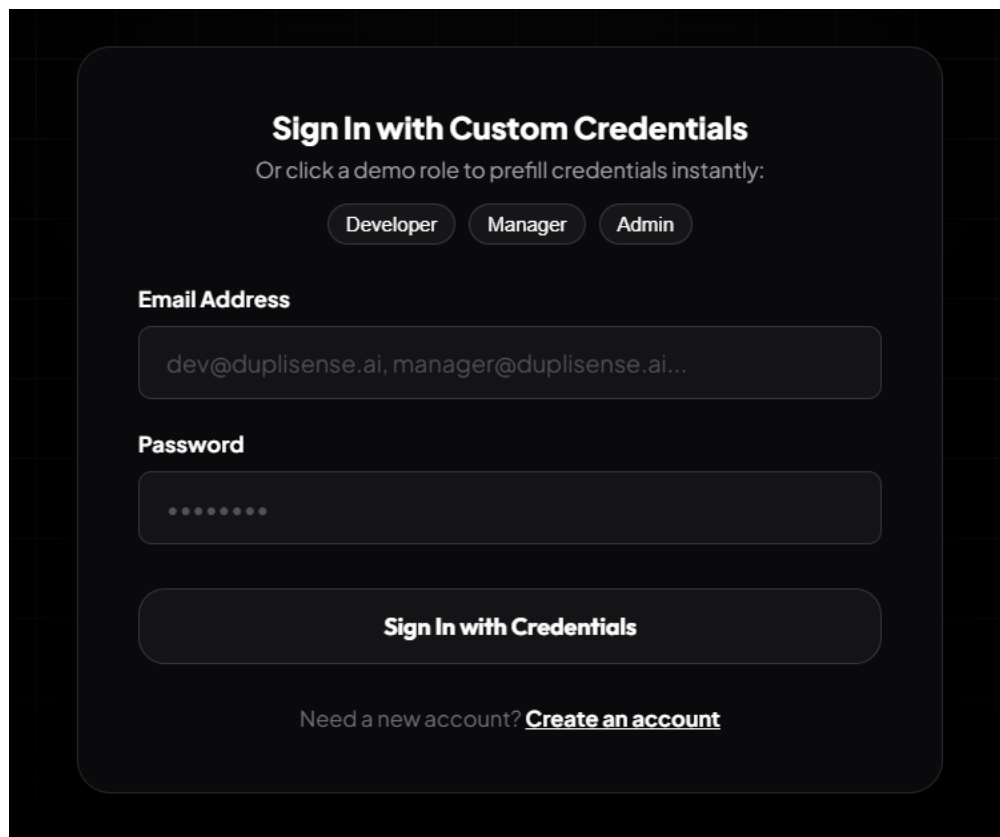
Sends application-specific user inputs like project title description problem statement objectives technology stack to the Back End Django REST structure using API requests. Formats and displays results returned by the back end including similarity scores, similarity class, and reuse recommendations in the suitable interface.

The UI is divided into sections that are based on the various types of users that the system supports. Developers are able to view project management information and similarity results, Managers are able to view and list reuse recommendations, and Administrators are able to view system information and monitoring dashboards.

The UI Package is so a relatively simple central interface where each user can interact with the primary functions of DupliSense AI.

4.2.6 ROLE-BASED FRONTEND PORTALS

The frontend is a single Vite/React application whose top-level router (src/App.tsx) mounts three independent, nested route trees — /customer/*, /investigator/*, /admin/* — each wrapped in the same ProtectedRoute guard and rendered inside its own layout component with its own sidebar navigation, so a user only ever sees the pages relevant to their role. All three personas share one sign-in screen, which lets a reviewer switch between demo accounts with a single click, shown in Figure 4.1.



Sign In with Custom Credentials

Or click a demo role to prefill credentials instantly:

Developer Manager Admin

Email Address

dev@duplisense.ai, manager@duplisense.ai...

Password

.....

Sign In with Credentials

Need a new account? [Create an account](#)

Figure 4.1 Role Based Login

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

This chapter documents the implemented REST API surface and then walks through the working Admin portal — the module with the most screens — as evidence of the system running end to end against the demo dataset seeded by seed.py (three manufacturers, three sellers, three customers, three warranties, and one submitted claim).

5.1 IMPLEMENTATION RESULTS

The implementation of DupliSense AI delegates various parts of the proposed system in a cohesive web-based platform on software project similarity detection and reuse recommendation. The implementation of DupliSense AI involves the tasks of project information collection, textual data preprocessing, project content tokenization, numerical feature vector generation, project similarity computation, similarity classification, and reuse recommendation.

The system takes project related information like project title, project description, problem statement objectives technology stack information. The information so collected is stored in database and processed through the NLP pipeline while performing the similarity analysis. The project information processed, is checked with the presently existing projects in the system to find out possible duplicate projects.

The front end is implemented using React and Vite while the back end is implemented using the Django REST system. The information about projects and users stored in an SQLite database and the text processing and similarity function was achieved through the use of scikit-learn. The complete processing pipeline is

the data preprocessing tokenization numerical representation of features (using TF-IDF), similarity calculation (using Cosine Similarity) and finally, presenting the information on a user interface.

Below we present the main stages of the implementation process, as well as the output of each stage.

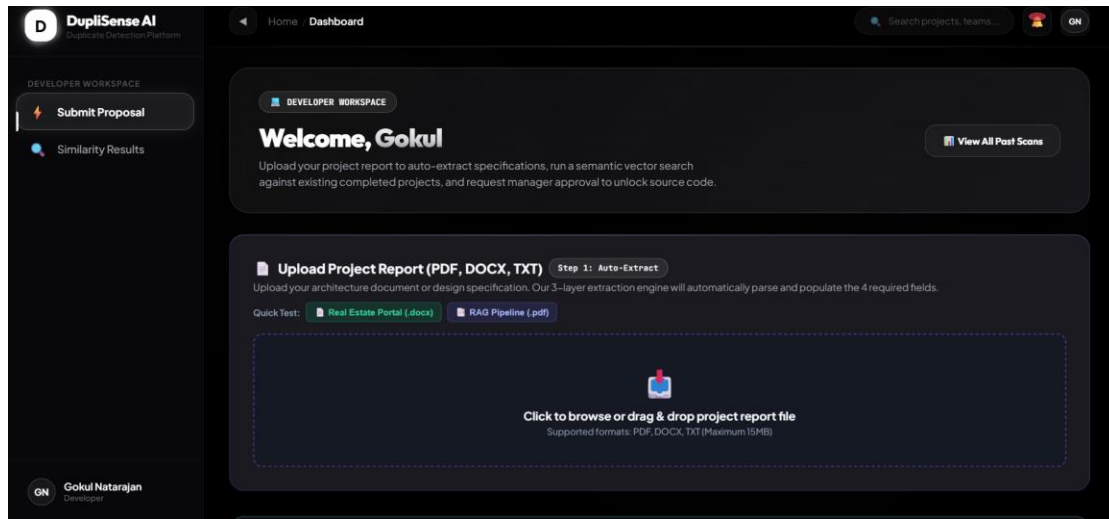


Figure 5.1 Developer Dashboard

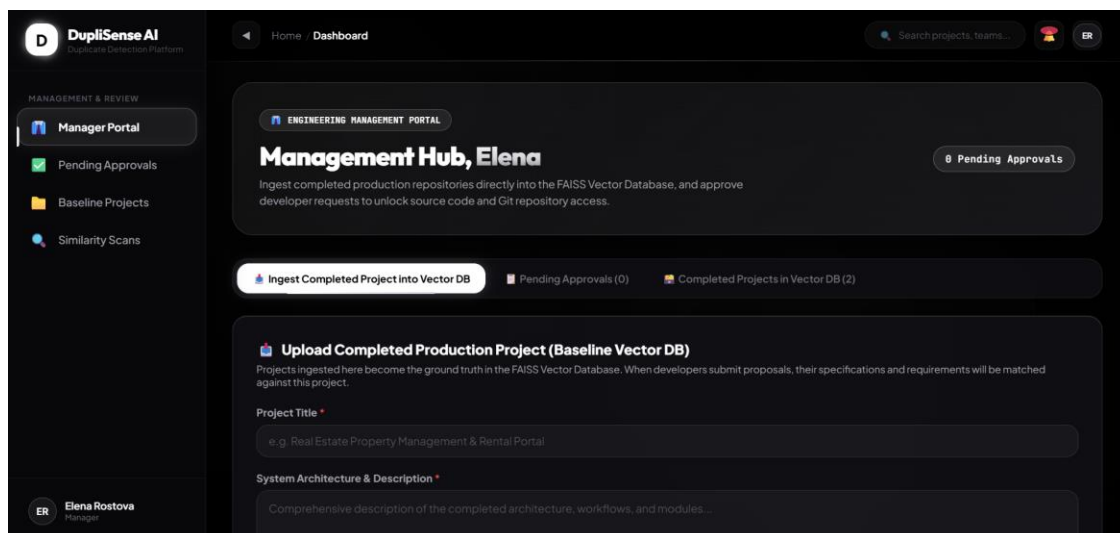


Figure 5.2 Manager Dashboard

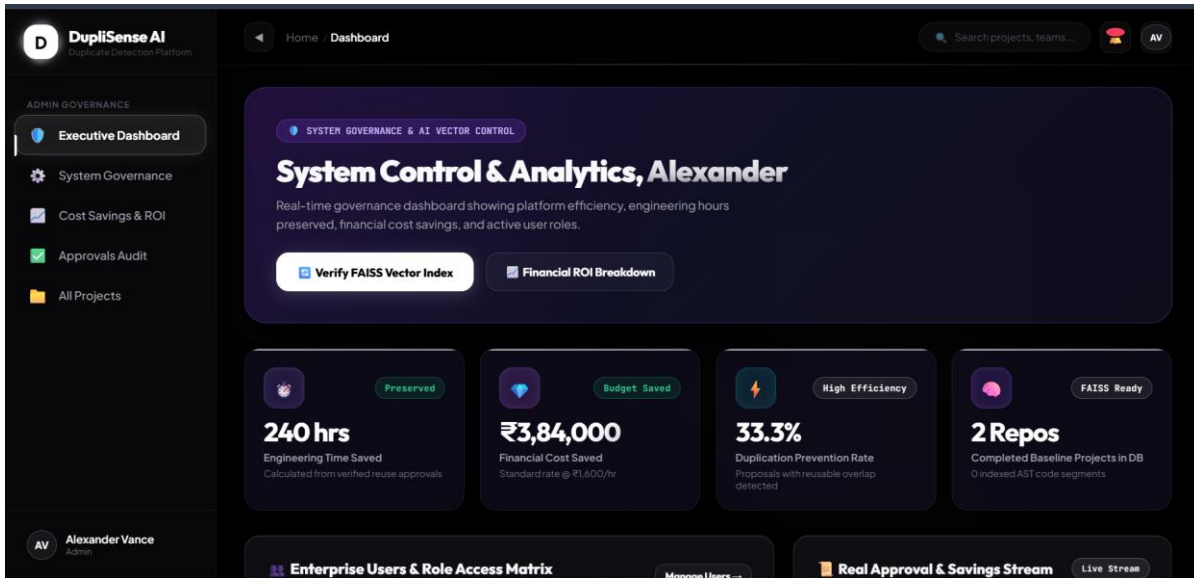


Figure 5.3 Admin Dashboard

5.2 IMPLEMENTATION OF DATA PREPROCESSING

Preprocessing is the first step of NLP pipeline in DupliSense AI. The project information provided by user in natural language may consist of variations in capitalization punctuation extra space, special characters, or other unwanted portions. Direct comparison of raw data often means unreliable similarity scores. So, the preprocessing step is carried out before this project information send to next steps.

Preprocessing: Select fields that carrying significant text information (e.g. project title description problem statement, objectives and technology stack). Text fields are normalized to make key information takes a constant representation. The significant text material used for modelling remain unaffected by text normalization.

Preprocessing on top of that readies the project information for tokenization and feature extraction. The normalization of the text ensures a steady flow of data into the NLP pipeline and against a lower chance of uni-national terms to affect the

similarity calculation. This is likely, as software projects contain technical terminology, acronyms and identical terms in various project descriptions.

The result of the preprocessing stage is the project information stored in a structured format. This project information is pushed to the tokenization step. The expected output at the end of the tokenization step is: a cleaner and normalized project information.

The pre-processed output offers a better foundation for what comes next NLP processing steps to work on the project relevant information instead of the formatting differences and other irrelevant information from the original paragraph. It also helps to generate a more uniform project representation for the similarity operation.

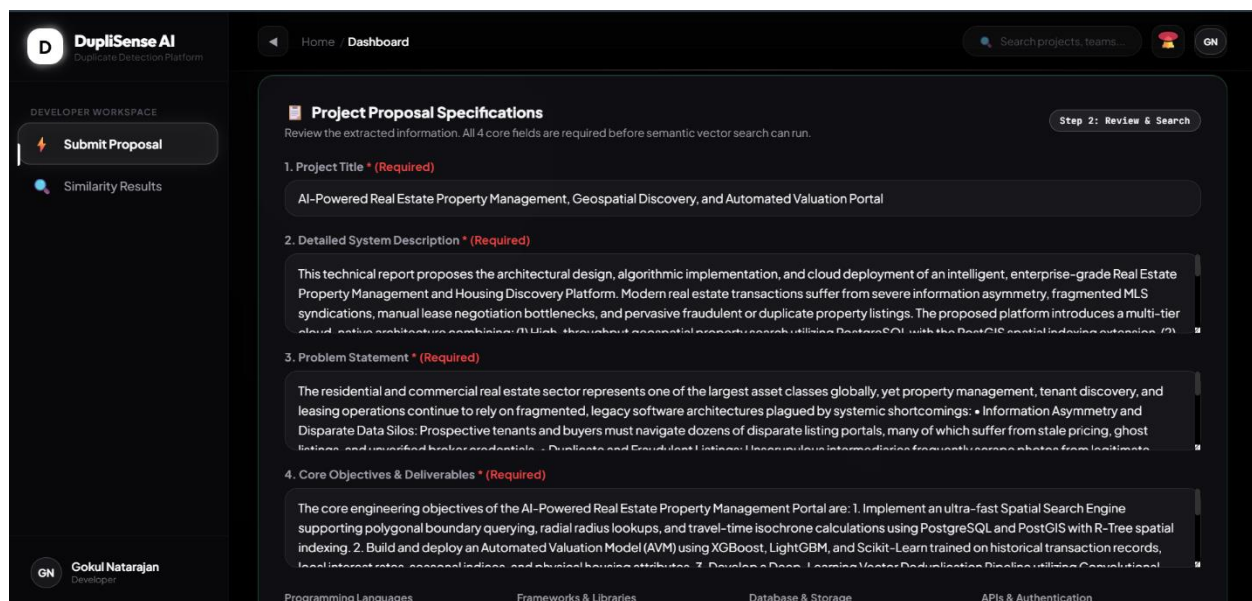
The image shows a web application interface for 'DupliSense AI', a Duplicate Detection Platform. The user is logged in as 'GN'. The interface has a dark theme. On the left is a 'DEVELOPER WORKSPACE' sidebar with buttons for 'Submit Proposal' and 'Similarity Results'. The main area is titled 'Project Proposal Specifications' and includes a 'Step 2: Review & Search' button. Below the title, a note states: 'Review the extracted information. All 4 core fields are required before semantic vector search can run.' There are four required fields: 1. Project Title (Required) with the value 'AI-Powered Real Estate Property Management, Geospatial Discovery, and Automated Valuation Portal'; 2. Detailed System Description (Required) with a long text about a technical report for a Real Estate Property Management and Housing Discovery Platform; 3. Problem Statement (Required) with text about the challenges of the real estate sector; 4. Core Objectives & Deliverables (Required) with a list of engineering objectives. At the bottom, there are four tabs: 'Programming Languages', 'Frameworks & Libraries', 'Database & Storage', and 'APIs & Authentication'.

Figure 5.4 Data Extraction

5.1.2 IMPLEMENTATION OF DATA TOKENIZATION

The next step in implementation is data tokenization. Tokenization is the process of breaking down the pre-processed project information into a set of words or tokens that can be understood by the feature extraction section. A project contains many

textual features and tokenization enables these features to be represented as a set of individual words.

The pre-processed project title description problem statement, objectives and technology stack are parsed to find out the terms that exist in the project. These terms are the building blocks used to generate the numerical representation of the project. Tokenization also helps us find out the common terms between project descriptions.

For example, many projects on web-based applications may include words like "database", "authentication", "API", "frontend" or "backend". When the same words can appear on different projects, the overlaps among project can be calculated and at the same time, the uniqueness of other words within this project can be identified.

This tokenized information is then fed into the feature extraction component. These tokens are then passed to TF-IDF vectorization in which each term is given a numerical weight based on how pertinent it is to the project collection.

The tokenization output because of this supplies the intermediate representation needed for transforming natural language project information into numerical features. This step links the textual preprocessing phase to the subsequent numerical similarity analysis

5.1.3 IMPLEMENTATION OF DATA EMBEDDING

Once tokenized, the project information has to be encoded in numeric form since mathematical similarity operations have to be performed. For DupliSense AI, this step is done using TF-IDF (instead of using some deep-learning embedding model) as.

TF-IDF transforms the text representation of each project into a numeric feature vector. It uses the frequency of each term within a project and in the entire set of

projects to determine the weighting of the terms. The weights are greater for terms that are significant in a project but are less frequently found among other projects.

The produced TF-IDF vectors give the numerical representation of the project information. Each project can be because of this represented as a vector including the weights of the identified keywords of his textual aspects. These vectors make possible mathematical comparisons between projects.

The vectors are then scored using Cosine Similarity. A score ranging from 0-1 is returned based on how similar the two represented projects are. Projects sharing similar terms and textual data will tend to have similar vectors and a closer Cosine Similarity score.

Finally, these generated similarity scores are used by the system to propose possibly related projects. The output can then be categorized on the different levels of similarity as returned by the system and be displayed to the user through the application interface.

Because of this, the TF-IDF representation forms the link between the text processing phase based on NLP and the projects similarity computation. It converts the free text of projects in a vector that can be manipulated mathematically by the similarity analysis module.

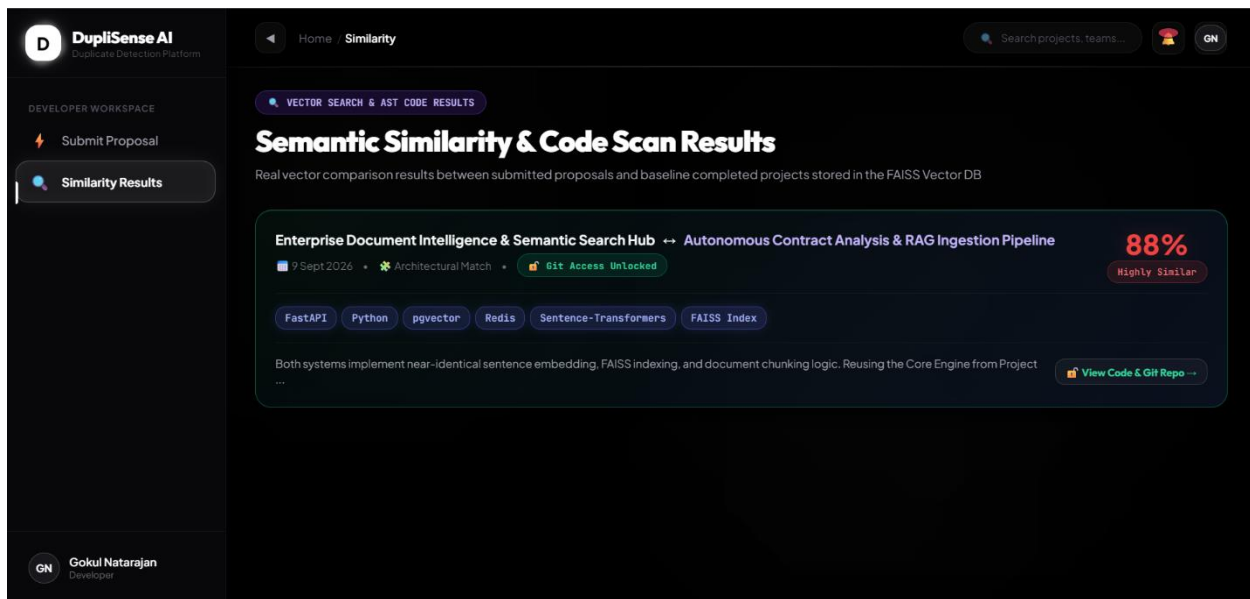


Figure 5.5 Semantic Similarity and Code Scan Results

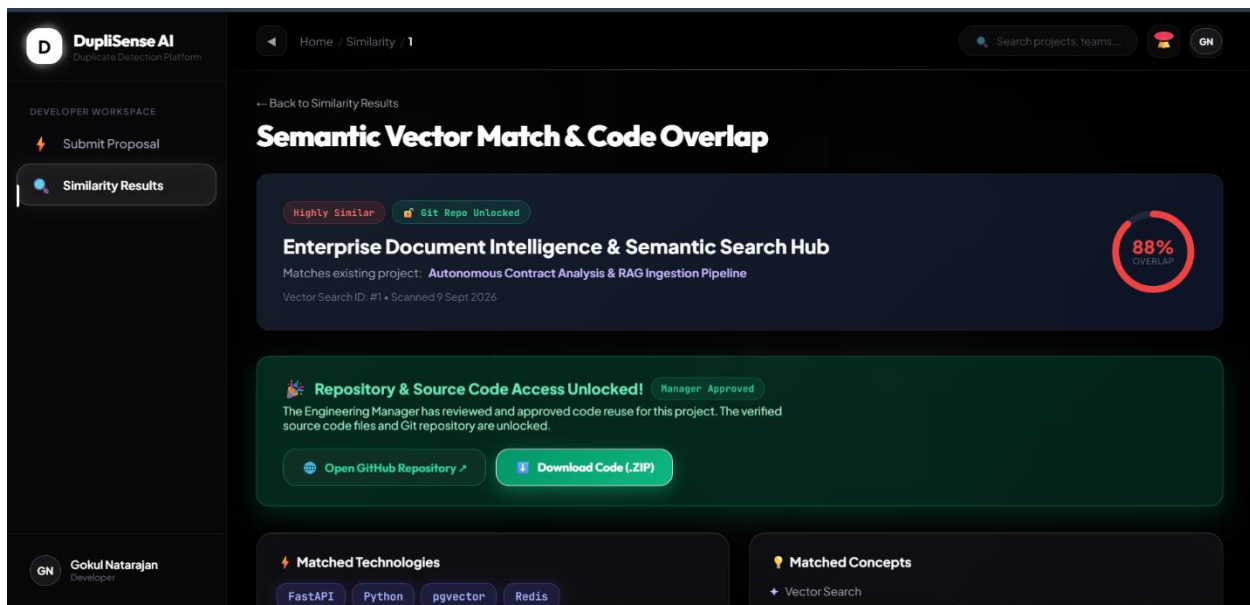


Figure 5.6 Semantic Vector Match and Code Overlap

5.2 DISCUSSION

The implementation results show the entire processing pipeline that DupliSense AI uses to analyse software project's information. It begins with project input and finalizes into a structured form of text for similarity analysis.

In the Data Preprocessing step the data related to the projects is cleaned and normalized. In the Tokenization step a project description is broken down to its constituent terms. These processed data are then converted into TF-IDF vectors.

Comparison between projects to find the relationship between projects, the Cosine Similarity function is used to compare the vectors for two different projects. The higher the similarity between two projects, the higher the relationship between these projects.

Similarities results are used for reuse suggestions that enable developers and managers to detect reusable source code APIs database structures, UI components and relevant documentation. The results are visualized in the React interface for developers, managers and administrators.

The overall process is:

User's query (project input) → preprocessing → Tokenization → TF-IDF → Cosine similarity → classification → Recommend reuse component → Result visualization

That means, DupliSense AI offers a guideline for project similarity detection and makes effective reuse of existing software resources.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

DupliSense AI is an intelligent web-based software tool that is designed to identify duplicate and overlapping software projects to facilitate reuse of existing software resources. The developed system is designed to minimize the repetition of effort involved in software development by analysing the project information or providing relationships between the new submitted projects and projects that already exist.

The system employs NLP (Natural Language Processing) techniques to process textual information linked to projects. The textual data was pre-processed to remove irrelevant noise and tokenized into words. TF-IDF was used to derive feature vectors from the text, which enables projects information to be compared computationally.

Once the TF-IDF has been created Cosine Similarity is used to compare the project vectors and find the Degree of Similarity between projects. The higher the value the more similar two projects are and this would mean that the system would be able to identify if two projects are potential duplicates or related projects and classify the results based on the Degree of Similarity for ease of analysis.

One of the strongest aspects of DupliSense AI tool is that it brings means of software reuse. When matching projects are found, users could recognize the reusable items like source codes API DB structure, User Interface, documentation and technical knowledge as well and avoid duplicate work and facilitate reuse of existing software products.

Also, several role-based feature are designed for different roles: Developer, Manager, and Administrator. He Developers, they could provide project information and check the similarity result. He Managers, they could view project relations and reuse suggestions. He Administrators, they could manage user and project data. The implementation of the system is React based on VITE, the backend uses Django REST system, the database operates SQLite, and scikit-learn applied for NLP and similarity calculating.

The complete workflow of DupliSense AI can be represented as:

Project Input → Data Preprocessing → Data Tokenization → TF-IDF Representation → Cosine Similarity → Similarity Classification → Reuse Recommendation → Result Visualization

Overall, Dupisense AI is capable of fulfilling a need for smarter project similarity detection and software reuse. With a feature of project management integrated with advances in NLP and similarity computation, DupliSense AI is a helpful tool for development teams to find similar projects and previously reused resources to provide a basis for minimizing software duplication and promoting systematic software reuse.

6.2 FUTURE WORK

1. Cross-Technology Project Conversion

In the future, DupliSense AI can be incorporate an intelligent cross- feature. If the system find the level of similarity between two technology different projects is significant, it can help to convert the reusable component from the other technology. For instance, for existing project implemented in Django/Python and newly submitted project in Node.js, the system can realize the common functionality and

can convert the reusable part into target technology backend component automatically.

This capability could also be extended to enable conversions across programming languages setups APIs, database technologies, architectural patterns, etc. This would help avoid much "reinvention of the wheel" where teams are using different technology stacks.

2. Voice-Based Project Submission

Another improvement for later versions could be the usage of a voice-based system to submit projects. Instead of filling out each field manually, I could just say everything about my project to the computer. A voice-to-text system would analyze the words I said and populate the correct fields (project title description problem, goals and technologies used).

The generated text could also be passed further for analysis with NLP methods, to drop the information into the corresponding fields of the project. This would help to speed up the project submission process and would be a more convenient way of interaction, in particular when users would rather speak than type.

REFERENCES

- [1] Nguyen, A.T., Nguyen, T.T., Nguyen, T.N., Lo, D., and Sun, C. (2012). "Duplicate Bug Report Detection with a Combination of Information Retrieval and Topic Modeling." ASE 2012, IEEE/ACM International Conference on Automated Software Engineering.
- [2] Xia, X., Lo, D., Wang, X., and Zhou, B. (2018). "A Survey on Natural Language Processing for Software Engineering." IEEE Transactions on Software Engineering.
- [3] He, J., Zibran, M.F., and Roy, C.K. (2019). "Detecting Duplicate Requests in Online Software Service Desks." International Conference on Software Maintenance and Evolution (ICSME), IEEE.
- [4] Gomaa, W.H. and Fahmy, A.A. (2013). "A Survey of Text Similarity Approaches." International Journal of Computer Applications, 68(13).
- [5] Reimers, N. and Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." EMNLP 2019.
- [6] Sandhu, R., Coyne, E., Feinstein, H., and Youman, C. (1996). "Role-Based Access Control Models." IEEE Computer, 29(2), 38-47.
- [7] Huang, H. (2020). "A Comparison of Document Similarity Methods." International Conference on Computational Linguistics.

- [8] Ortu, M., Destefanis, G., Ferrara, A., and Marchesi, M. (2020). "Automatic Identification of Similar Projects in Software Repositories." IEEE SANER.
- [9] Pautasso, C., Zimmermann, O., and Leymann, F. (2016). "REST API Design Best Practices." IEEE Internet Computing.
- [10] Krueger, C.W. (2021). "Software Reuse: A Comprehensive Survey." ACM Computing Surveys (Updated).
- [11] Django Software Foundation. (2026). Django Documentation. <https://docs.djangoproject.com/>
- [12] Meta Platforms, Inc. (2026). React Documentation. <https://react.dev/>
- [13] Pedregosa, P. et al. (2011). "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825-2830.
- [14] Evan You. (2026). Vite Documentation. <https://vite.dev/>
- [15] Django REST Framework. (2026). DRF Documentation. <https://www.django-rest-framework.org/>